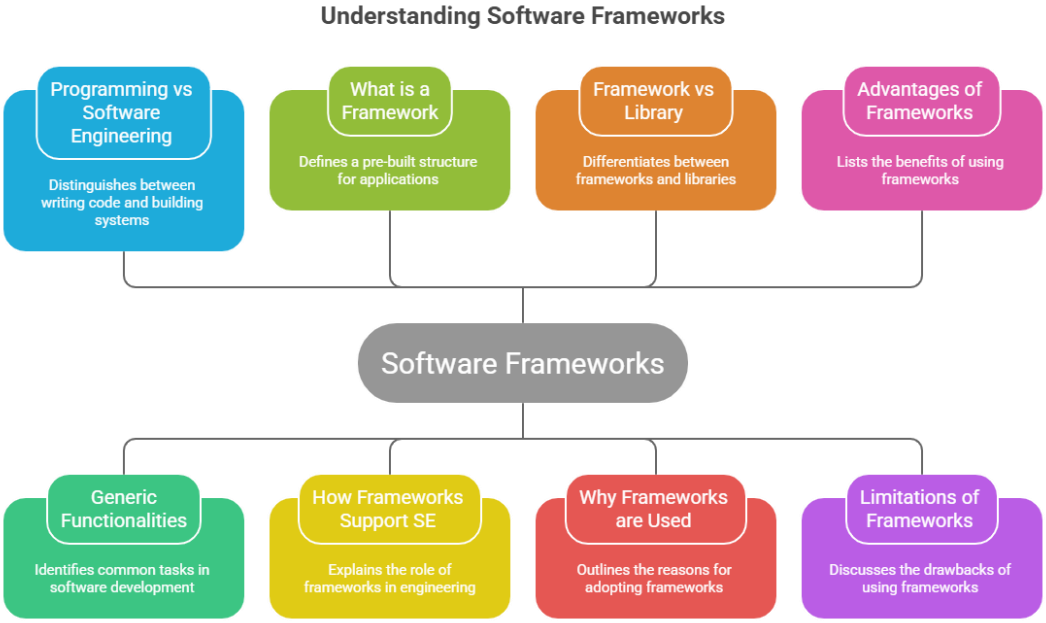
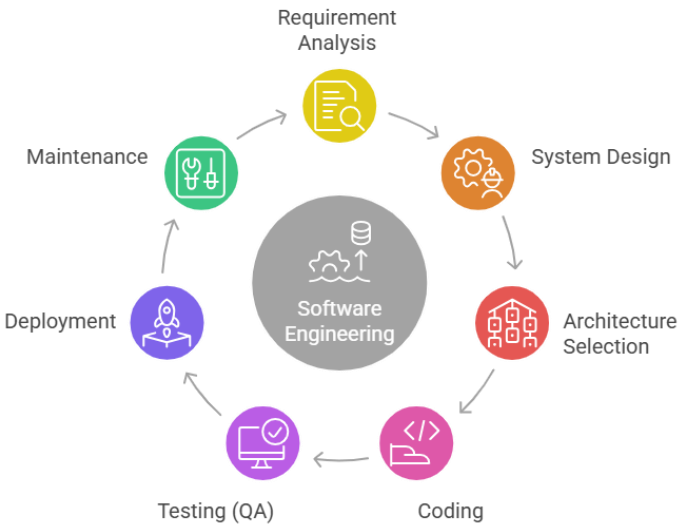

IT3030
Programming
Applications
and Frameworks
Lecture Short Notes
Learn with SACHI

Contents

Lecture 01 - Software Frameworks: An Overview



Aspect	Programming	Software Engineering
Definition	Writing code to create a program or application	The complete process of designing, developing, testing, and maintaining software
Focus	Coding and implementing logic	Managing the entire software development lifecycle
Scope	Small part of software creation	Covers the whole software development process
Activities	Writing code, debugging	Requirement analysis, system design, coding, testing, deployment, maintenance
Goal	Make the program work	Build reliable, scalable, and maintainable software
Example	Writing Java, Python, or JavaScript code	Building a banking system using proper design, testing, documentation, and maintenance
Role	Programmer / Developer	Software Engineer / Development Team



Generic Functionalities in Software Development

Most applications share **common features**. Examples: User authentication (login) / Routing / Database connection /Scheduling jobs

Example

Imagine building **100 websites**. Each site needs: a login system, a database connection, and user sessions. Without frameworks, you must rewrite everything again and again.

Solution: Reusable components. This is why **frameworks exist**.

What is a Framework: It is a reusable architecture containing software components used to build applications faster. Framework = **pre-built structure**

Language	Framework
Java	Spring Boot
JavaScript	Angular
NodeJS	Express
Python	Django
PHP	Laravel
C#	.NET

How Frameworks Help Software Engineering

Frameworks provide:

- **Architectural Patterns (**MVC / MVVM)
- **Design Patterns (**Factory pattern / Singleton pattern) **-** Reusable design solutions.
- **Inversion of Control** - Framework controls program flow instead of the developer.
- **Standards (**Naming conventions / Folder structures / Coding standards)
- **Tooling Support** (Dependency managers / Package managers / Code generators)
- **Testing Support (** Unit testing **/** Integration testing)

Aspect	Library	Framework
Program Flow Control	You control the program flow	Framework controls the program flow
Usage	You call the library when needed	Framework calls your code
Code Integration	Library functions are used inside your code	Your code is inserted into the framework
Control Principle	Normal control flow	Inversion of Control (IoC)
Example	import lodash and use its functions	Using frameworks like React, Angular, or Django

Why Use Frameworks

Frameworks help **reduce development time** because they provide many built-in tools and structures that developers can directly use. Instead of building everything from scratch, developers can rely on the framework's existing components.

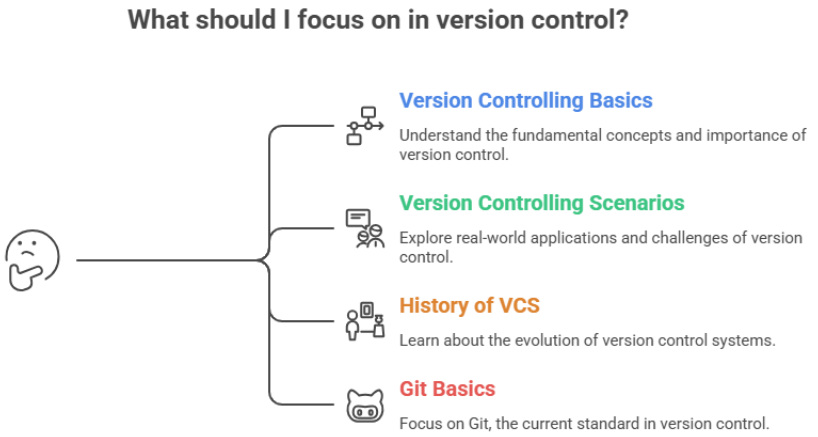
Because of this, developers can mainly focus on **business logic and application features**, rather than spending time on **low-level coding** such as basic system setup, routing, or configuration. This makes the development process **faster, more organized, and efficient**.

Aspect	**Advantages of Frameworks**	**Limitations of Frameworks**
Development Speed	Faster development because many features are already built-in	Learning the framework rules and structure takes time
Code Efficiency	Code reuse through reusable components and modules	Limited flexibility due to predefined structure
Debugging	Easier debugging because frameworks provide tools and error handling	Some frameworks include unnecessary features that increase complexity
Support	Strong community support with tutorials, documentation, and updates	Choosing the wrong framework may reduce system performance
Security	Better security standards with built-in protections	Developers must frequently update when new versions are released
Code Structure	Less boilerplate code and more organized project structure	Framework conventions must be strictly followed

Software Engineering Process

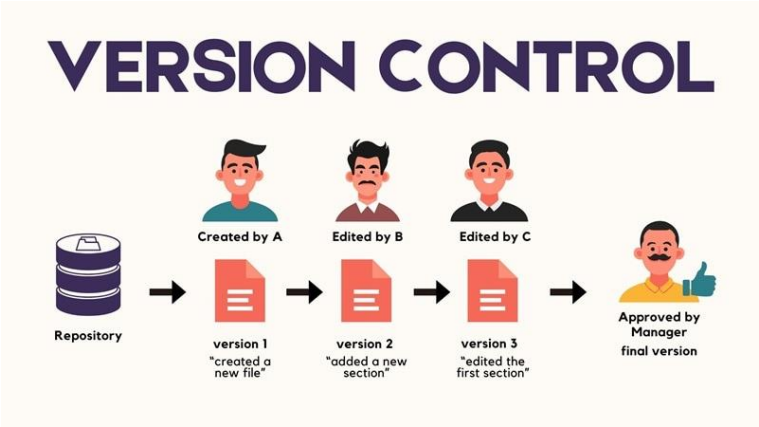
- **Requirement Analysis**
- **System Design**
- **Architecture Design**
- **Implementation (Coding)**
- **Testing**
- **Deployment**
- **Maintenance**

Lecture 02 - Version Controlling & Workflows with Git



Without version control, you may save files like: project_final.doc , project_final_v2.doc , project_final_last.doc , project_final_really_last.doc**. This becomes confusing. That is why Version Control Systems (VCS) exist.**

Version control : is a system that records changes to a file or set of files over time so that you can recall specific versions later



Manual Version Control:

Advantages	Disadvantages
Files are safely stored and backed up	High risk of accidentally overwriting files
Can access files from anywhere	Difficult for multiple people to collaborate
	Hard to track changes across versions
	No proper history, making rollback tricky

VCS Type / Generation	Example	Features / Key Points	Advantages	Limitations
Manual Version Control		Files managed manually, no automation	Files stored safely, Accessible anywhere	Risk of overwriting, Difficult collaboration, Hard to track changes, No proper history
First Generation – Local VCS	SCCS (1973)	Works on a single computer, files tracked locally		Only one developer, No team collaboration
Second Generation – Centralized VCS	SVN, CVS	Client-server architecture, central repository, multiple developers	Central repository, supports teams	Server failure stops the project, requires internet
Third Generation – Distributed VCS	Git	Peer-to-peer, full repository per developer, works offline	Works offline, Reliable, Modern standard	
Modern VCS Examples	Git, SVN, Mercurial	Automated version tracking, history management, and collaboration	Easy collaboration, Tracks history, Multiple developers	Learning curve for tools requires proper setup

Git was created by **Linus Torvalds** in 2005 for **Linux kernel development**. It is a **distributed version control system**, which supports **parallel branches**, **offline commits**, and **team collaboration**.

Git Basic Concepts

Local Repository

Repository stored on developer's computer.

Contains:

- project files
- commit history

Remote Repository

Shared repository for the team.

Examples:

- GitHub
- GitLab
- Bitbucket

Git ≠ GitHub

Git = version control system

GitHub = cloud hosting service for Git repositories.

Git Workflow (Basic Flow)

Steps:

- Add file to repo
- Modify file
- Commit changes
- Pull updates from remote
- Push changes to remote

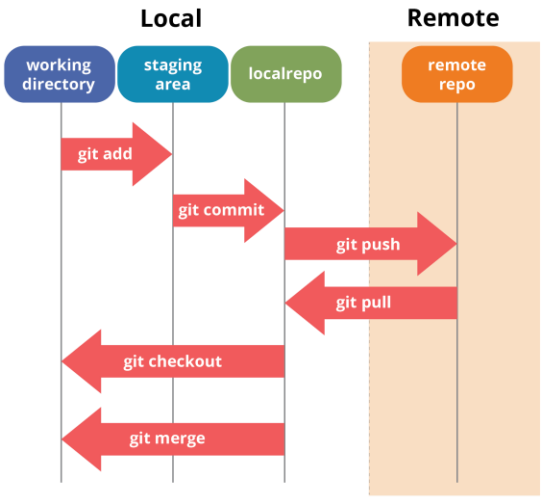
Command	Purpose
git add	Add file to staging area
git commit	Save snapshot
git push	Upload changes
git pull	Download updates
git merge	Merge branches
git checkout	Switch branch

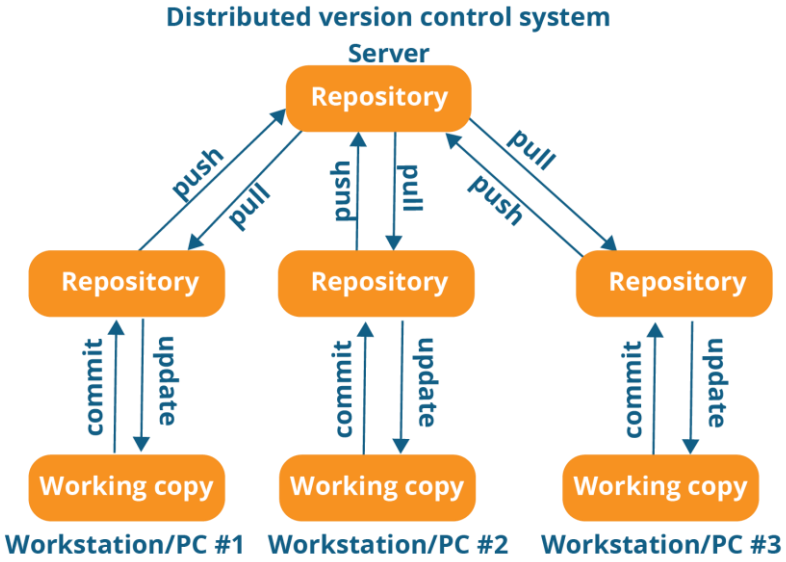
A **commit = snapshot of changes**.

Each commit has:

- **Commit ID**
- **SHA-1 hash**

HEAD : It refers to **latest commit in current branch**.



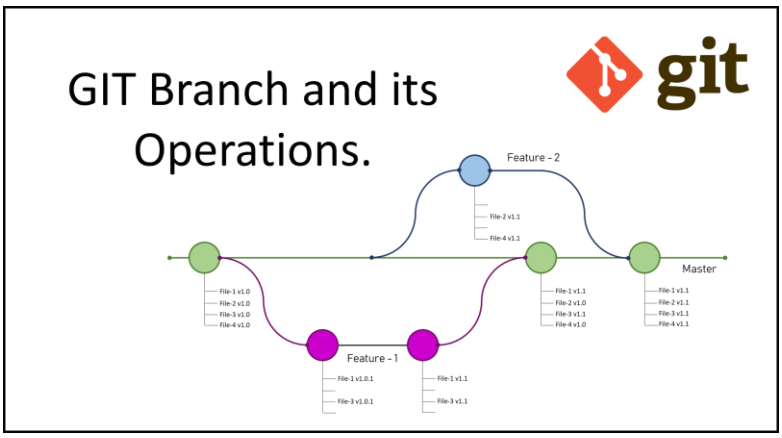


Lecture 03 - Version Controlling & Workflows with Git – II

Git Branch : A **branch** is a separate line of development.
It allows developers to:

- Work on features independently
- Avoid affecting the main project code

The **main branch** contains stable code while new work happens in other branches.



Why branches are important

✓ Parallel development

✓ Avoid breaking main code

✓ Multiple developers can work simultaneously

Creating and Switching Branches

Common Git commands:

Command	Purpose
git branch branch-name	Create branch
git checkout branch-name	Switch branch
git checkout -b branch-name	Create + switch branch

Git Merge: It combines changes from one branch into another.
Example: **feature-login → main** Command: **git merge feature-login**
This integrates the new featu re into the main project.

Merge Conflict
A **merge conflict occurs when two developers modify the same part of a file.**
Example: Developer A changes line 10 , Developer B also changes line 10
Git cannot decide which version to keep. So a **conflict occurs**.

Resolving conflicts
Steps:

- Git shows conflicting code
- Developer manually chooses the correct version
- File is saved and committed again

Git Workflow (Team Collaboration)
A **Git workflow** defines how developers collaborate.
Example workflow:

1. Clone repository
2. Create branch
3. Make changes
4. Commit changes
5. Push branch
6. Create Pull Request
7. Review code
8. Merge into main branch

This workflow ensures safe collaboration.

Pull Request (PR): It is a request to merge changes from one branch into another.
It allows team members to:

- Review code
- Discuss changes
- Suggest improvements

Pull requests are used in platforms like:

- GitHub
- GitLab
- Bitbucket

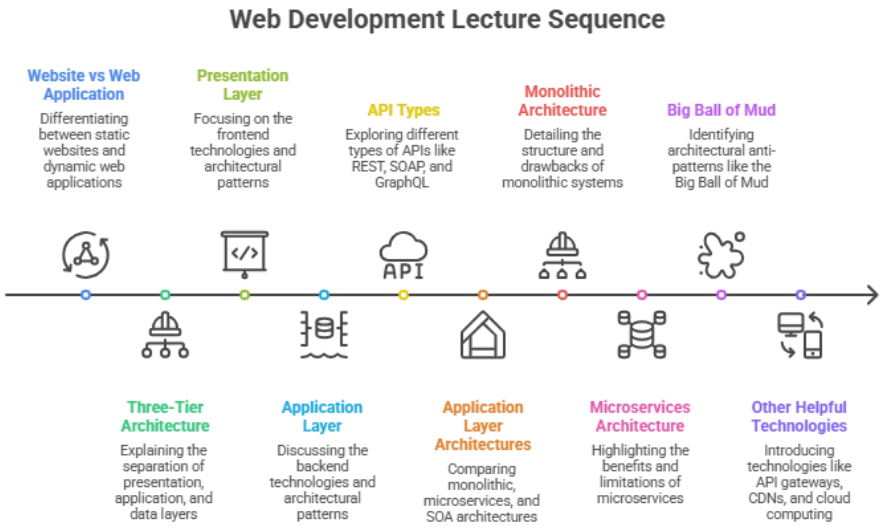
Forking Workflow: It is mainly used in **open-source projects**.

Steps:

1. Fork repository
2. Clone fork locally
3. Create branch
4. Make changes
5. Push to fork
6. Submit pull request to original repository

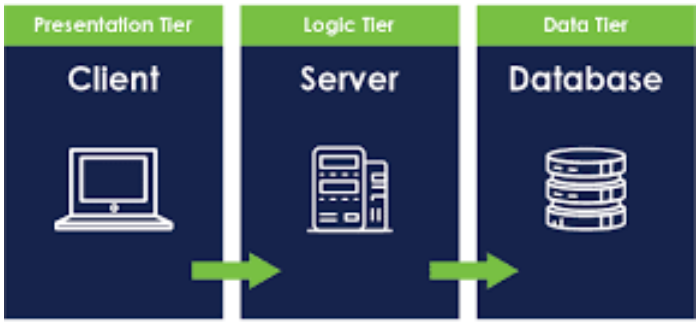
This allows contributors to propose changes without direct access.

Lecture 04 – Web Application Architecture



Aspect	Website	Web Application
Main Purpose	Provide information/content	Provide interactive functionality
Content Type	Mostly static	Dynamic and interactive
User Role	View / read only	Input, process, receive output
Interaction Level	Low	High
Processing	Very little user-based processing	User requests are processed
Example	News site , Blog , Portfolio	Facebook , Online banking , E Commerce Site

Three-Tier Architecture is a way of designing a web application by dividing it into 3 separate layers



Three-Tier Architecture

- Presentation Layer (Frontend)
- Application Layer (Backend)
- Data Layer (Database)

Layer	Role	Also Called
Presentation Layer	User interface	Frontend / Client-side
Application Layer	Business logic	Backend / Server-side
Database Layer	Data storage	Data layer

Advantages of Three-Tier Architecture

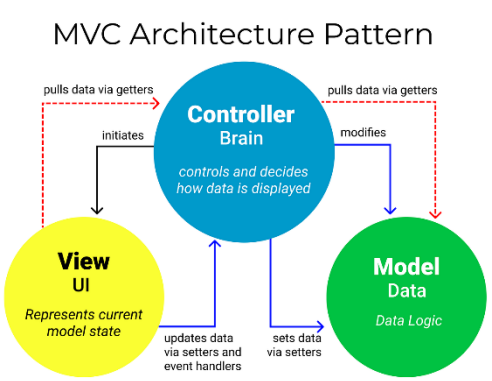
✓ Each layer runs independently

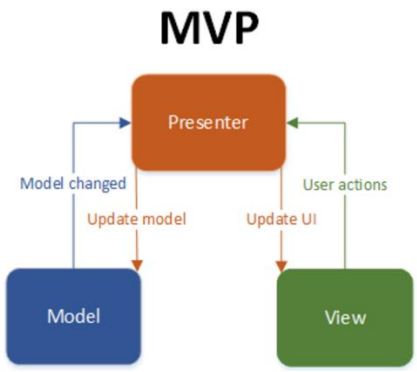
✓ Parallel development (different teams)

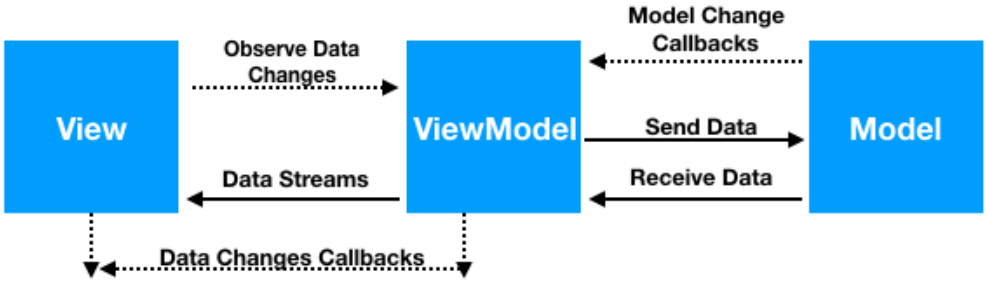
✓ Independent scaling

✓ Better security (data is isolated)

Aspect	Presentation Layer	Application Layer	Database Layer
Main Purpose	Displays the user interface and handles user interaction	Processes requests, applies business logic, and controls application behavior	Stores, retrieves, and manages application data
Also Called	Frontend / Client-side	Backend / Server-side	Data Layer / Database Tier
Main Users	End users directly interact with this layer	Developers and server-side systems mainly work here	Database administrators and backend systems mainly interact here
What It Handles	Pages, forms, buttons, layouts, user input, display of results	Validation, calculations, authentication, routing, API handling, business rules	Data storage, queries, updates, deletion, persistence
Input	User actions such as clicks, typing, form submissions	Requests from frontend or other services	Queries from backend
Output	Visible content to the user	Processed responses, API results, business decisions	Requested data or confirmation of data changes
Examples of Tasks	Show login page, display product list, collect form data	Check login details, process order, calculate totals, send API response	Save user account, fetch products, store orders
Technologies Mentioned in Lecture	HTML, CSS, JavaScript, React, Angular, Vue, jQuery, AJAX	Java with Spring/Spring Boot, Node.js with Express/Koa, Python with Django/Flask, PHP with Laravel/Symphony, .NET	Databases such as relational or other storage systems used to keep application data
Architecture / Patterns	MVC, MVVM, MVP, Component Architecture, Micro Frontends	Monolithic Architecture, , SOA	Usually accessed through queries and data models from backend
Communication	Sends requests to backend and shows responses to users	Receives requests from frontend, communicates with database, returns results	Responds to backend data requests
Direct User Interaction	Yes	No	No
Business Logic	No or very little	Yes, this is the main place	No
Data Storage	No	Temporary processing only	Yes, permanent storage happens here
Security Role	Basic UI-level validation and access display	Main security rules, authentication, authorization, request handling	Protects stored data and access permissions
Scalability	Can scale UI layer separately	Can scale business logic separately	Can scale storage separately
Independent Development	Yes	Yes	Yes
Benefit in 3-Tier Architecture	Better user experience and clear UI separation	Keeps logic organized and reusable	Keeps data isolated and secure
If This Layer Fails	Users cannot properly see or use the app	Main functions stop working	Data cannot be stored or retrieved
Example in Online Shopping App	Product page, cart page, checkout form	Price calculation, order processing, login checking	Product table, user table, order records

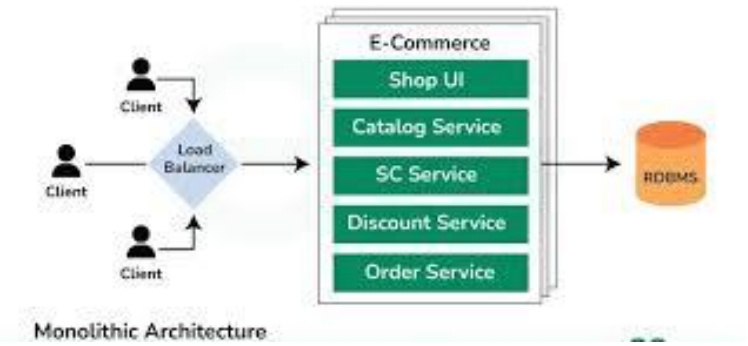


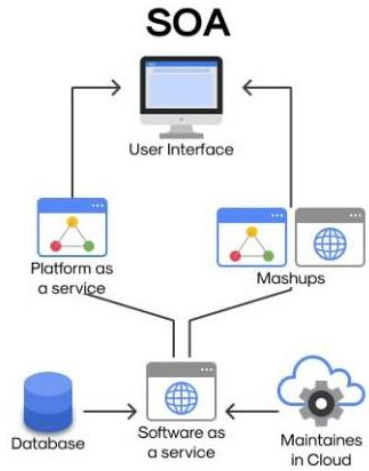


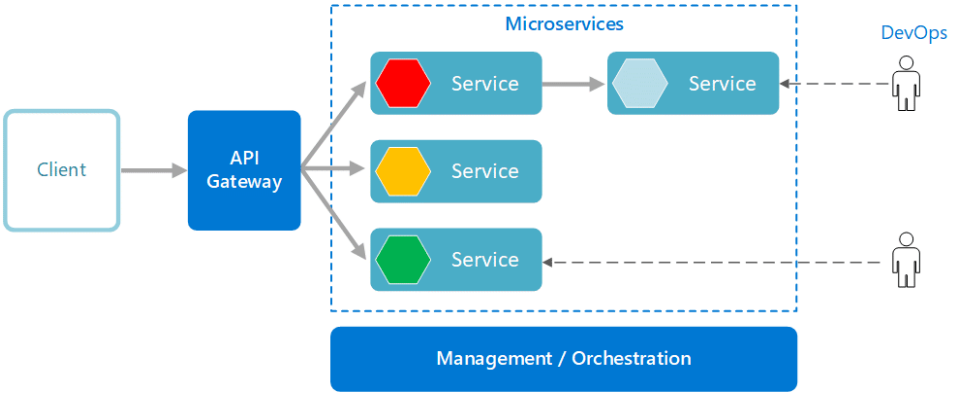


Aspect	MVC	MVVM	MVP
Full Form	Model View Controller	Model View ViewModel	Model View Presenter
Main Idea	Separates app into Model, View, and Controller	Separates app into Model, View, and ViewModel	Separates app into Model, View, and Presenter
Model	Handles data and business logic	Handles data and business logic	Handles data and business logic
View	Displays UI to user	Displays UI to user	Displays UI to user
Third Component	Controller handles input and updates Model/View	ViewModel connects View and Model	Presenter connects View and Model
User Input Goes To	Controller	View, then ViewModel	View, then Presenter
View Talks To	Controller / sometimes Model updates reflected	ViewModel	Presenter
Controller / ViewModel / Presenter Role	Controls flow between Model and View	Exposes data and commands for View	Handles presentation logic and updates View
Coupling	View and Controller are related	Better separation between View and logic	View and Presenter communicate closely
Data Binding	Usually not automatic	Strong support for data binding	Usually no automatic data binding
Best Use Case	Traditional web apps	Modern UI frameworks with binding	Apps needing clear testable presentation logic
Complexity	Moderate	Can be more complex at first	Moderate
Testability	Good	Very good	Very good
Common In	Traditional server-side apps, many web frameworks	Angular, WPF, modern frontend frameworks	Android older patterns, UI-heavy apps
Main Advantage	Simple and widely used	Easier UI synchronization through binding	Clear separation and easy unit testing
Main Limitation	Controller can become large	Harder to understand initially	More boilerplate than MVC sometimes

Aspect	Monolithic Architecture	Microservices Architecture	SOA (Service-Oriented Architecture)
Basic Idea	Entire application is built as one single unit	Application is built as many small independent services	Application is built as reusable services that communicate over a network
Structure	One large codebase	Many small codebases / services	Multiple services connected through service communication
Service Size	Large, tightly connected system	Small, focused services	Services are usually larger than microservices and business-oriented
Coupling	Tightly coupled	Loosely coupled	Loosely coupled
Deployment	Whole application must be deployed together	Each service can be deployed independently	Services can often be deployed independently
Scalability	Hard to scale one part only	Easy to scale individual services	Services can be scaled separately depending on design
Development Speed	Can become slow as app grows	Faster for separate teams on separate services	Moderate, depends on service design and integration
Flexibility	Low flexibility	High flexibility	High flexibility
Technology Choice	Usually one language / one framework stack	Supports polyglot programming, different services can use different technologies	Can also support multiple technologies
Reliability	Failure in one module may affect whole app	Failure in one service may not crash whole system	More reliable than monolith if services are separated well
Maintenance	Harder as application becomes larger	Easier to maintain each small service	Easier than monolith, but integration can be complex
Complexity Type	Low at start, high local complexity later	High global/system complexity	High integration and communication complexity
Debugging	Easier at first because everything is in one place	Harder because many services interact	Hard because multiple services and integrations are involved
Infrastructure Cost	Lower compared to microservices	Higher infrastructure cost	Often high due to middleware and service communication needs
Communication	Internal function/module calls	API/network communication between services	Service communication, often through middleware or enterprise service mechanisms
Best For	Small to medium apps, simple systems, early-stage projects	Large, complex, scalable applications	Enterprise systems needing reusable business services
Main Advantage	Simple to build and start	Independent deployment and scaling	Reusable services across enterprise systems
Main Disadvantage	Hard to scale and maintain when large	Complex to manage, costly, hard to debug	Can become heavy and complex in integration/governance
Example Memory	One big app	Many small apps working together	Business services shared across systems







API = software interface that allows communication between systems

Key Ideas

✓ Acts as a **contract**
✓ Hides complexity
✓ Enables integration

Benefits

✓ Faster development
✓ Reuse existing services
✓ Better collaboration

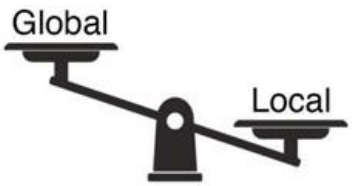
Types of APIs

✓ REST
✓ SOAP
✓ RPC
✓ WebSockets
✓ GraphQL

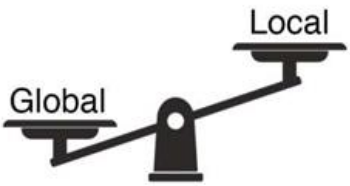
Big Ball of Mud (Anti-pattern) - Poorly structured system with no clear architecture
Key Idea: High complexity / Hard to maintain

Other Important Technologies

✓ API Gateway
✓ CDN
✓ Caching (Redis)
✓ Load Balancer
✓ Message Queue
✓ Cloud Computing
✓ Cloud Storage
✓ Virtual Machines



Big Ball of Mud



Distributed Big Ball of Mud

Lecture 05 – REST APIs

Consuming Web Resources

Type	How they access web
Humans	Web browsers, apps
Programs	APIs

Programs use **Web APIs to communicate**

API (Application Programming Interface)

API = **software interface that allows communication between applications**

Key Ideas

- Hides complexity
- Acts as a **contract**
- Enables integration

API Types

Type	Description
SOAP	Protocol-based (strict)
RPC	Function call style
REST	Most popular modern approach
CORBA	Legacy

What is REST?

REST = **architectural style for building web services** Introduced by **Roy Fielding (2000)**

Key Points

- Not a protocol
- Not a technology
- Uses HTTP
- Flexible

Why REST is Popular

Reason	Explanation
Simple	Easy to learn
Flexible	No strict rules
Lightweight	Less overhead than SOAP
Widely used	Modern standard

REST Basic Concepts

Term	Meaning
Client	Requests data
Server	Provides data
Resource	Data/content
Representation	JSON/XML format

REST uses **Request → Response model**

REST Design Principle

Strong **decoupling between client & server**

Server sends:

- Resource data
- Links (URIs)

Six REST Constraints

1. Client-Server - Separate client and server

✓ Improves scalability

✓ Independent development

2. Stateless - Each request is independent

✓ No session stored on server

✓ Request must contain all info

3. Cacheable - Responses can be cached

✓ Improves performance

✓ Reduces server load

4. Uniform Interface

4 Sub-Constraints

Sub-constraint	**Meaning**
Self-descriptive messages	Request contains enough info
Resource identification	Uses URI
Representation	Data (JSON/XML)
HATEOAS	Links to next actions

5. Layered System - System has multiple layers

✓ Supports proxies, load balancers

✓ Improves scalability

6. Code on Demand - Server can send executable code

✓ Optional constraint

True REST vs RPC

NOT REST if:

- Only uses HTTP methods
- Only uses pretty URLs
- No hyperlinks

Key Difference

Feature	**REST**	**RPC**
Hypermedia	Yes (HATEOAS)	No
Structure	Resource-based	Action-based
Discovery	Dynamic	Hardcoded

Without HATEOAS → NOT TRUE REST

Richardson Maturity Model - Measures how RESTful an API is

Levels

Level	Description
Level 0	Single endpoint (POX)
Level 1	Multiple URIs
Level 2	Uses HTTP methods (GET, POST)
Level 3	HATEOAS (Fully RESTful)

HTTP Methods

Method	Meaning
GET	Read
POST	Create
PUT	Update
DELETE	Delete

HTTP Status Codes

Code	Meaning
2XX	Success
4XX	Client error
5XX	Server error

HTTP Headers

- Content-Type
- Authorization
- Cache-Control

JSON & HAL

JSON - Lightweight data format / Key-value pairs

HAL - Format for **RESTful responses with links**